

Uncertainty-gated selection for block-sparse attention*

A value-of-information view of the SSA-style selector

Thomas Rossi
Eonpass
t.rossi@eonpass.com

June 30, 2026

Abstract

Block-sparse attention scales long-context language models by replacing the $O(N^2)$ softmax with a per-query top- k selection over key blocks. This cutoff is myopic: when the k -th and $(k+1)$ -th blocks are nearly tied in score, the selector commits without spending extra budget, and a dropped block carrying answer evidence is unrecoverable downstream. We propose a value-of-information *router* that measures, for each query, how decisively the top- k cut was made, and doubles the kept set for the queries where that gap is smallest; the rule is backbone-agnostic and stacks with existing block-scoring methods such as Quest. On LongBench-v2 medium at $n = 215$ (the entire dataset subset), router-on-Quest reaches paired recall 0.75 vs. top- k 0.47 – +28 pp over the SSA-style baseline (McNemar $p < 0.01$) – and lands within 2 pp of dense on RULER NIAH multikey at the same context. The lift reproduces on four models from three architectures (Qwen2.5, Mistral-Nemo, Qwen3.6). At 128K, the router preserves 0.81 and 0.89 of dense accuracy on Qwen2.5-7B-1M and Qwen3.6 (vs. SSA-style top- k at 0.09 on the former) while the fused selection-plus-kernel pipeline runs at $0.62\times$ and $0.80\times$ dense wall time.

1 Introduction

Long-context language models increasingly use *block-sparse attention* as a drop-in replacement for the $O(N^2)$ softmax. The idea is shared by Quest (Tang et al., 2024), H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), MInference (Jiang et al., 2024), NSA (Yuan et al., 2025), MoBA (Lu et al., 2025), and Subquadratic’s SSA (Subquadratic, 2025): a cheap per-query *selector* picks k out of N_B key blocks; exact attention then runs only on the selected set. The selector is the lever – it decides where the model attends. Today it is universally implemented as a *single-shot top- k rule* on a pooled inner product $q \cdot \bar{k}_b$ between the query and a per-block key summary.

This is a myopic decision. When the k -th and $(k+1)$ -th block are nearly tied, top- k silently breaks the tie and moves on; if the dropped block contained an evidence token, the answer is gone, and no downstream layer can recover it. The failure mode is structural, not noisy: it bites hardest on *multi-hop* and *query-latent* retrieval, where the relevance of a block depends on what was learned earlier in the same forward pass and is not visible to the selector’s surface query–key match. SSA’s own reported NIAH multi-key recall degrades sharply as the number of keys grows (Subquadratic, 2025).

*Code, data, and reproduction scripts: <https://github.com/ThomasRossi/uncertainty-gated-block-sparse-attention>. Persistent archive: [doi:10.5281/zenodo.20630587](https://doi.org/10.5281/zenodo.20630587) (concept DOI; always resolves to the latest version).

This paper. We treat the top- k cutoff as a *value-of-information* (VoI) decision and add a single layer of policy on top of it. For each Q-tile and head, we compute the normalised cutoff margin

$$\sigma = \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \in [0, 1],$$

where $s_{(\cdot)}$ are the sorted block scores. A small σ means the cutoff is high-risk; we then *route* that tile to a $2\times$ expanded `kv_idx` – it gets to attend to more blocks – while confident tiles keep the baseline k_{budget} . The expansion is paid for selectively: we trigger on the bottom q -fraction of tiles per layer, so the average attended set grows by only $1 + q$ blocks per row.

The router is backbone-agnostic. The cutoff margin σ is a function of the sorted block scores; it does not depend on how those scores are computed. Existing selectors differ in their block-scoring backbone – SSA pools keys with a mean ($\bar{k}_b = \frac{1}{B_n} \sum_j k_j$), Quest uses a min/max upper bound ($s = \sum_d \max(q_d k_{b,d}^{\max}, q_d k_{b,d}^{\min})$). The router can sit on top of any of them. This turns the contribution from *a replacement for top- k* into *a generic budget-allocation layer that stacks on whichever scoring backbone is best for the task*. We validate this empirically: **better scoring (Quest) and better budget allocation (router) are orthogonal directions; combining them strictly dominates either alone** on both benchmarks we test.

Contributions.

1. A VoI formulation of the selector cutoff that adds one scalar per tile and one quantile threshold per layer, independent of the block-scoring backbone. No retraining, no extra parameters, no per-row keep tensor.
2. Empirical demonstration that the router composes with two distinct scoring backbones (SSA-style K-mean and Quest’s K-min/K-max) and that, on every model in the panel, the router-lifted version of the *winning* backbone dominates the unlifted version of the other. The result holds on two standardised benchmarks (RULER NIAH multi-key and LongBench-v2 medium), four models from three architecture classes (Qwen2.5-14B-Instruct, Qwen2.5-7B-Instruct-1M, Mistral-Nemo-Instruct-2407, Qwen3.6-35B-A3B), and contexts from 32K to 128K. Headline: router-on-Quest paired recall **0.75** vs top- k 0.47 on LongBench-v2 medium at $n = 215$ (+28 pp, McNemar $p < 0.01$). Cross-architecture finding: on QK-Norm models (Qwen3.6), K-mean dominates K-max and *router-on-K-mean* replaces router-on-Quest as the headline composition – the scoring-backbone choice is data-dependent on the trained attention distribution, but the router lifts whichever wins. Full panel in Section 4.
3. A fused selection-plus-kernel implementation that produces `kv_idx` directly per Q-tile and handles the routed expansion in the same code path, keeping kernel dispatch shape-uniform. All four sparse policies (top- k , router, Quest, router-on-Quest) run on this same kernel; they differ only in the selection step. Wall-time profile crosses dense at 32K and widens at 64K ($0.76\times$ – $0.85\times$); the crossover regime is characterised via an Amdahl decomposition of the prefill.
4. A custom diagnostic benchmark, the *Pointer-Chase Haystack* (PCH; Appendix B), used during method development to isolate selector quality from model capability.
5. A negative-control result (LongBench-v1) that pins down *when* the router helps: only when the selector’s per-query budget (the blocks it keeps) is small relative to where the answer-relevant evidence lives.

2 Background and related work

2.1 Block-sparse attention

A standard decoder layer (Vaswani et al., 2017) maps hidden state $h^{(L)} \in \mathbb{R}^{N \times d_{\text{model}}}$ to $h^{(L+1)}$ via

$$h' = h^{(L)} + \text{Attn}(\text{LN}(h^{(L)})), \quad h^{(L+1)} = h' + \text{MLP}(\text{LN}(h')).$$

The attention block, for query position i on head h , computes

$$\text{Attn}(x)_{i,h} = \sum_{j \leq i} \frac{\exp(q_{i,h} \cdot k_{j,h} / \sqrt{d_{\text{head}}})}{\sum_{j' \leq i} \exp(q_{i,h} \cdot k_{j',h} / \sqrt{d_{\text{head}}})} v_{j,h}.$$

Block-sparse attention replaces $\{j \leq i\}$ by a small selected subset S_i , chosen per query at inference time on frozen weights.

2.2 The selector landscape

Concrete selectors differ in (a) how they pool keys into a per-block summary and (b) how they score blocks against the query, but **they all reduce, at the per-query selection step, to a top- k rule over a block score**:

- **SSA** (Subquadratic, 2025) (Subquadratic Sparse Attention) – per-Q-tile top- k over mean-pooled keys, $s_b = q \cdot \bar{k}_b$ with $\bar{k}_b = \frac{1}{B_n} \sum_j k_j$. Cheap to compute; mean-pooling blurs single hot keys, which is exactly the multi-key NIAH degradation discussed above.
- **Quest** (Tang et al., 2024) – per-block elementwise $K_b^{\min}, K_b^{\max}$ summaries; score is an upper bound on $\max_{j \in b} q \cdot k_j$, computed coordinate-wise (see Eq. 2). Recovers sharp single-key needles that mean-pooling loses.
- **H2O** (Zhang et al., 2023), **SnapKV** (Li et al., 2024), **MInference** (Jiang et al., 2024) – top- k over learned or attention-history-based block scores.
- **NSA** (Yuan et al., 2025), **MoBA** (Lu et al., 2025) – end-to-end learned gating, but still resolves to a top- k over block scores at the per-query selection step.

What none of these methods do is treat the cutoff as a *decision under uncertainty*: when $s_{(k-1)} \approx s_{(k)}$, the selector commits without spending extra budget. That step is the lever this paper pulls. Because the lever is on the cutoff (not on how s is computed), it composes with any of the scoring backbones above. In the experiments we evaluate it on top of both the SSA-style K-mean backbone and Quest’s K-max upper bound.

2.3 Long-context evaluation

The published benchmarks fall into two families. **Synthetic / diagnostic**: RULER (Hsieh et al., 2024) (NIAH, VT), BABILong, MRCR. Designed for controlled stress tests of long-context recall. Their failure modes are interpretable but they are *not* predictive of downstream performance, as HELMET (Yen et al., 2024) has documented. **Real-task**: LongBench (Bai et al., 2023, 2024), HELMET (Yen et al., 2024), NoCha. These cover multi-hop QA, summarisation, code, and so on. LongBench-v2 in particular has a *medium* split with native prompt lengths typically $\geq 100\text{K}$ words, designed to stress long-context selection. We use **RULER NIAH** (synthetic, standardised) and **LongBench-v1 + v2 medium** (real-task, standardised) as the headline benchmarks, with a custom diagnostic benchmark (**PCH**, Appendix B) used during method development.

3 Method

We describe the working method end-to-end. Each step is motivated by the previous one and lifts a specific weakness. The full per-equation derivation lives in Appendix A; here we give the path the reader needs to follow to understand the experiments.

3.1 Step 1: block scoring

Keys are grouped into contiguous blocks of $\text{BLOCK}_N = 64$ tokens. We evaluate the router on top of two block-scoring rules.

The first is the SSA-style mean-pooled-key inner product: at layer L , the selector scores block $b \in \{0, \dots, N_B - 1\}$ via

$$s^{(L)}[i, h, b] = \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \quad \bar{k}_b^{(L,h)} = \frac{1}{\text{BLOCK}_N} \sum_{j \in \text{block } b} k_j^{(L,h)}. \quad (1)$$

Mean pooling blurs single-key signals: a block containing one hot key adjacent to noise scores like a block of all noise, and gets dropped.

The second is Quest’s K-min/K-max upper bound (Tang et al., 2024), which replaces $\bar{k}_b$ with an elementwise pair $(K_b^{\min}, K_b^{\max})$ and scores

$$s_b^{\text{quest}} = \sum_{d=1}^{d_{\text{head}}} \max(q_d \cdot K_{b,d}^{\max}, q_d \cdot K_{b,d}^{\min}), \quad (2)$$

a coordinate-wise upper bound on $\max_{j \in b} q \cdot k_j$ that preserves the single-key signals the mean averages away.

We treat the choice between Eqs. (1) and (2) as a *backbone hyperparameter*; everything downstream (per-tile selection, the cutoff margin σ , the trigger) is identical.

3.2 Step 2: per-tile selection

A naive per-row top- k emits a boolean keep tensor of shape $[B, H, M, N_B]$ that downstream must sort and union across rows of a Q -tile; this becomes the dominant cost at long context.

Following the SSA recipe, we group $\text{BLOCK}_M = 64$ consecutive query rows into a Q -tile t and make one selection decision per tile, shared by all rows in that tile:

$$\text{tile_score}[t, h, b] = \max_{r \in \text{tile } t} s[r, h, b], \quad \text{kv_idx}[t, h] = \text{top-}k(\text{tile_score}[t, h, \cdot]). \quad (3)$$

The sink block 0 and the tile’s own block are forced into kv_idx by adding $+\infty$ to their scores *before* the top- k , which avoids a downstream concat-and-dedup. The output is a single integer tensor of shape $[B, H, Q_t, k_{\text{budget}}]$ (where $Q_t = M/\text{BLOCK}_M$), passed straight to the attention kernel without a per-row inner mask. Wall time for selection scales as $O(NN_B/\text{BLOCK}_M)$ rather than $O(NN_B)$ for the per-row version.

This is coarser than per-row by construction: a block that one row in the tile strongly wants can be outranked by a block that several rows weakly want, since the tile score is a max over rows of an inner product. The router (Steps 3–6) addresses this – it spends a controlled amount of extra budget on the tiles where the top- k cut was ambiguous, while leaving confident tiles untouched.

3.3 Step 3: the cutoff is a decision – read its uncertainty

The top- k in Eq. (3) is a decision: keep block $(k-1)$, drop block (k) . Its *quality* is naturally measured by how decisive that ranking is. Let $s_{(0)} \geq s_{(1)} \geq \dots$ be the sorted tile scores for tile t , head h . We define the normalised cutoff margin

$$\sigma[t, h] = \frac{s_{(k-1)} - s_{(k)}}{s_{(0)} - s_{(k)}} \in [0, 1]. \quad (4)$$

The interpretation is direct:

- $\sigma \rightarrow 1$ – the kept set is far above the rejected tail; the cutoff is unambiguous.
- $\sigma \rightarrow 0$ – the kept set’s last element is tied with the first rejected element; the cutoff is a coin flip.

σ is computed from a top- $(k+1)$ partial sort – one extra element beyond the top- k that produces kv_idx – at no extra asymptotic cost.

Why this is a value-of-information signal. If we were to commit to the top- k , the expected loss from the cutoff decision is bounded by the probability mass the softmax would have placed on the dropped block. When the k -th and $(k+1)$ -th scores are close, that mass is large; when they are well-separated, it is small. The cutoff margin σ is a cheap, dimensionless proxy for this expected loss. The formal anchor is in Appendix D: under a sub-Gaussian noise model on the block scores, σ is the dimensionless analogue of the asymptotically optimal best-arm-identification exploration index of Garivier and Kaufmann (2016), and the σ -quantile router is $C(\tau)$ -times optimal in top- k identification, with $C(\tau) \leq 2$ under Gaussian noise.

3.4 Step 4: aggregate σ across heads

We aggregate the per-head margin $\sigma[t, h]$ to a per-tile signal via a uniform mean:

$$\bar{\sigma}[t] = \frac{1}{H} \sum_{h=1}^H \sigma[t, h]. \quad (5)$$

$\bar{\sigma}[t]$ is the tile’s overall decision confidence: low when many heads simultaneously face an ambiguous cutoff, high when most heads agree the cutoff is clean.

Aggregation is necessary: gating directly on the per-cell $\sigma[t, h]$ does not work (Appendix C). Heads in the same tile are highly correlated, and per-cell gating is dominated by per-head idiosyncratic noise that has no relation to whether the tile is actually ambiguous. Averaging across heads recovers the signal.

3.5 Step 5: trigger on the bottom q -fraction

We turn $\bar{\sigma}$ into a binary decision via a per-layer empirical quantile:

$$\text{trigger}[t] = \mathbb{1}[\bar{\sigma}[t] \leq \text{quantile}_q(\bar{\sigma}[\cdot])]. \quad (6)$$

q is the only hyperparameter introduced by the router; intuitively it is the per-layer fraction of tiles that are deemed risky enough to spend extra budget on. We use the same q across all layers and find it stable.

Choosing q . The working value $q = 0.40$ was selected from a sweep on RULER NIAH-multikey at $n = 30$, $\text{ctx}=32\text{K}$ (Appendix C), where paired recall rises monotonically with q over the tested range $\{0.10, 0.20, 0.30, 0.40\}$ and plateaus near 0.40. We use the same value unchanged across all benchmarks, models, and contexts in Section 4; the sweep was not repeated per task. A more thorough $\rho \times q$ Pareto study remains open (Section 4.5).

Why a quantile, not an absolute threshold. The absolute scale of σ varies across layers (early layers are more peaked than later layers). A quantile is the simplest layer-conditional normalisation. We did test absolute and per-cell quantile thresholds; they fail (Appendix C).

3.6 Step 6: expand kv_idx uniformly

Triggered tiles get a $\rho \times$ expanded top- k over tile_score (we use $\rho = 2$). The selection operation here is the same top- k that every selector in the literature reduces to; tile_score inherits whichever block-scoring backbone of §3 is in play (Eq. (1) or Eq. (2)), so the same expansion rule applies to both “router” and “router-on-Quest” without modification.

To preserve a fixed kernel dispatch shape across triggered and non-triggered tiles, we allocate $[B, H, Q_t, \rho k_{\text{budget}}]$ uniformly and pad non-triggered tiles’ extra slots with the kernel’s N_B -sentinel:

$$\text{kv_idx}[t, h] = \begin{cases} \text{top-}\rho k(\text{tile_score}[t, h, \cdot]) & \text{if trigger}[t] = 1, \\ \text{top-}k(\text{tile_score}[t, h, \cdot]) \parallel [N_B, \dots, N_B] & \text{otherwise.} \end{cases} \quad (7)$$

The kernel skips $b = N_B$ on a fast check, so non-triggered tiles cost the same as plain top- k . The end-to-end forward is otherwise unchanged: a standard FlashAttention-style online softmax, iterating each tile’s kv_idx instead of all N_B blocks.

Cost accounting. The average kept blocks per row is $k_{\text{budget}}(1 + q(\rho - 1))$. At $q = 0.4, \rho = 2$ this is $1.4 k_{\text{budget}}$. The measured wall-time cost is sub-proportional to this block-count factor: router/top- k kernel-time ratio is ≈ 1.53 at 32K and ≈ 1.47 at 64K (Section 4.3).

4 Experiments

4.1 Setup

Models. All weights frozen; no retraining, no fine-tuning. We evaluate on four instruction-tuned models spanning three architecture classes (dense Qwen2.5, dense Mistral, hybrid+MoE Qwen3.6):

- *Qwen2.5-14B-Instruct* – 40 Q / 8 KV heads, $d_{\text{head}}=128$, 48 layers, trained at 32K context. The headline model.
- *Qwen2.5-7B-Instruct-1M* – 28 Q / 4 KV heads, $d_{\text{head}}=128$, 28 layers, trained at 1M context. Same family as the headline model, but a smaller parameter count and a much longer training window; used for 128K-context experiments (Section 4.3) since the 14B model’s 32K training window precludes them.
- *Mistral-Nemo-Instruct-2407* – 32 Q / 8 KV heads, $d_{\text{head}}=128$, 40 layers, trained at 128K context. Different architecture family, used to test cross-family transfer. The Mistral tokenizer requires

`fix_mistral_regex=True` on `AutoTokenizer.from_pretrained`; otherwise the load path is unchanged.

- *Qwen3.6-35B-A3B* – hybrid+MoE: 40 layers organised as (`linear_attn` $\times$ 3 $\rightarrow$ `full_attn` $\times$ 1) so only 10/40 layers run softmax attention; full-attention heads are 16 Q / 2 KV, $d_{\text{head}}=256$ with partial RoPE on the first 64 dims and per-head QK-Norm; MLP is sparse MoE (128 experts, top-8 activated; 3B active params of 35B total); trained at 128K context.

The router code path and the fused selection-plus-kernel implementation run unchanged across all four models; the only model-dependent code is the tokenizer flag above and, for Qwen3.6, two harness-side prompt tweaks (disabling `enable_thinking` on the chat template and prefilling the assistant turn with "The correct answer is (" for multiple-choice).

Hardware. A100-80GB via Modal for the three Qwen2.5 / Mistral-Nemo models; H200 for Qwen3.6-35B-A3B (does not fit on A100). The Qwen3.6 image additionally installs `flash-linear-attention` and `causal-conv1d` so that the 30 linear-attention (Gated DeltaNet) layers and their conv branch run on fused kernels.

Sparse path. Custom Triton block-sparse attention kernel with fused per-tile selection.

Baselines and policies.

- *dense* – FlashAttention (Dao et al., 2022) via PyTorch SDPA, the ceiling.
- *top-k* – per-tile top- k on the K-mean backbone (Eq. 1). SSA-style baseline; uses only Step 2.
- *Quest* – per-tile top- k on the Quest K-max upper-bound backbone (Eq. 2). Strong baseline on sharp-needle tasks.
- *router* – Steps 2–6 on top of the K-mean backbone, $q = 0.40, \rho = 2$. Uncertainty-gated expansion only.
- *router-on-Quest* – Steps 2–6 on top of the Quest K-max backbone, same q, ρ . Combines better scoring with budget allocation.

All four sparse policies share the same kernel and selection code; they differ only in (a) which backbone scores blocks and (b) whether the router expansion is applied. Wall-time and quality numbers are therefore directly comparable across them.

Selector budget. Fixed at $k_{\text{budget}} = 33$ blocks per tile, corresponding to roughly 2K attended tokens per row (the PCH diagnostic in Appendix B uses slightly different budgets, noted in-place).

Metrics. The primary metric is **accuracy** (or task score, e.g. F1 on extractive QA): the unconditional success rate – fraction of examples the policy answers correctly. This is the standard reporting unit in the long-context literature (Quest, H2O, SnapKV, MInference, NSA, SSA). Headline tables lead with it.

The secondary, diagnostic metric is **paired recall**: among the examples dense answers correctly, the fraction the sparse policy also answers correctly,

$$\text{paired recall} = \left| \{i : \text{dense}_i \text{ correct} \wedge \text{sparse}_i \text{ correct}\} \right| / \left| \{i : \text{dense}_i \text{ correct}\} \right|.$$

We write n for the total number of examples and $n_{dc} \leq n$ for the number dense answers correctly; n_{dc} is the denominator of paired recall, not n . We report paired recall alongside accuracy because raw accuracy conflates two failure modes – the selector dropped a needed block vs. the model could not have answered even with full attention – and paired conditions on $\{i : \text{dense}_i \text{ correct}\}$ to isolate the first.

On RULER NIAH dense accuracy is 1.00, so the two metrics coincide cell-by-cell; on LongBench-v2 they diverge (raw accuracy clusters tightly across sparse policies while paired recall ranges widely), and that divergence is itself analysed in Section 4.2.

Hit rate (fraction of gold-evidence blocks the selector keeps) is used only in the diagnostic experiments of Appendix B, where the gold blocks are known by construction.

4.2 Quality at 32K context across benchmarks

Setup. Two standardised long-context benchmarks at fixed context 32K and selector budget $k_{\text{budget}} = 33$, $q = 0.40$, $\rho = 2$, on all four panel models:

- *RULER NIAH-multikey* (3 keys), $n = 100$. Single-hot-key retrieval; dense answers $\geq 96\%$ of items on every model in the panel, so the accuracy ceiling is essentially 1.00 and accuracy coincides with paired recall cell-by-cell.
- *LongBench-v2 medium*, $n = 215$ (the entire medium-length subset of the dataset; no further filter). Multi-hop multiple-choice (A/B/C/D); native item length $\geq 100\text{K}$ words, middle-truncated to 32K so the selector budget binds. Dense accuracy varies across models from 0.16 to 0.41, so accuracy and paired recall diverge – both are reported.

Result.

Table 1: RULER NIAH-multikey at $n = 100$, $\text{ctx} = 32\text{K}$. Values are unconditional **accuracy**. Dense accuracy is ≥ 0.96 on every model so accuracy and paired recall coincide cell-by-cell (Nemo differs by ≤ 0.01 on Quest). Backbone column indicates the block-scoring rule (K-mean = Eq. 1; K-max = Quest, Eq. 2). “–” indicates a configuration not run.

policy	backbone	Qwen-14B	Qwen-7B-1M	Nemo-12B	Qwen3.6-35B-A3B
dense	–	1.00	1.00	0.96	1.00
top- k	K-mean	0.51	0.28	0.29	0.94
router	K-mean	0.63	0.39	–	0.98
Quest	K-max	0.93	0.94	0.50	0.85
router-on-Quest	K-max	0.98	1.00	0.66	0.97

Table 2: LongBench-v2 medium at $n = 215$, $\text{ctx} = 32\text{K}$. Each model column reports unconditional **accuracy** (acc) and **paired recall** (paired); dense paired is 1.00 by construction. n_{dc} on the dense row is the dense-correct count and the denominator of paired. Note the metric divergence: sparse accuracies cluster tightly near dense accuracy while paired recall ranges widely – the selector-quality signal lives entirely in the paired column.

policy	backbone	Qwen-14B		Qwen-7B-1M		Nemo-12B		Qwen3.6	
		acc	paired	acc	paired	acc	paired	acc	paired
dense	–	0.19	1.00	0.16	1.00	0.27	1.00	0.41	1.00
n_{dc}	–	40		35		58		88	
top- k	K-mean	0.19	0.47	0.16	0.60	0.24	0.50	0.41	0.90
router	K-mean	0.19	0.60	0.16	0.66	0.26	0.64	0.39	0.92
Quest	K-max	0.21	0.68	0.15	0.66	0.26	0.57	0.27	0.33
router-on-Quest	K-max	0.21	0.75	0.20	0.86	0.26	0.69	0.32	0.48

Read. Two observations across the panel:

(i) *The router improves the accuracy of whichever backbone it is applied to, on every model and benchmark in the panel.* On RULER NIAH: K-mean lift on Qwen-14B $0.51 \rightarrow 0.63$ (McNemar $p < 0.01$), Qwen-7B-1M $0.28 \rightarrow 0.39$, Qwen3.6 $0.94 \rightarrow 0.98$; K-max lift on Qwen-14B $0.93 \rightarrow 0.98$, Qwen-7B-1M $0.94 \rightarrow 1.00$, Nemo $0.50 \rightarrow 0.66$ ($p < 0.001$), Qwen3.6 $0.85 \rightarrow 0.97$. On LongBench-v2 the same pattern holds on both backbones across all four models (Table 2); the LB-v2 accuracy spread is small on Qwen2.5 + Nemo (saturated within standard error around dense 0.16–0.27) but materially separates on Qwen3.6 (0.27–0.41).

(ii) *Which backbone is the better one depends on the model.* On Qwen2.5 (both sizes) and Mistral-Nemo, the K-max (Quest) backbone dominates K-mean on both benchmarks, sometimes by very large margins (RULER NIAH: Qwen-14B Quest beats top- k by +42 pp; Qwen-7B-1M by +66 pp). On Qwen3.6 the ordering inverts: K-mean dominates K-max (RULER NIAH top- k 0.94 vs Quest 0.85; LB-v2 paired 0.90 vs Quest 0.33). The mechanism is structural: Qwen3.6 applies per-head RMSNorm to Q and K post-projection (QK-Norm), which regularises away the per-head magnitude variance the K-min/K-max upper bound exploits, so the Quest envelope flattens and mean-pool becomes at least as informative. Testable prediction: any future QK-Norm model should reproduce this reversal. Combining (i) and (ii), the best policy is router-on-the-winning-backbone, model by model: router-on-Quest on Qwen2.5 + Nemo, router-on-K-mean on Qwen3.6.

Paired recall as a selector diagnostic. The literature universally reports unconditional accuracy on these benchmarks; we add paired recall as a secondary, selector-isolated reading. On RULER NIAH dense accuracy is ≥ 0.96 so the two metrics coincide cell-by-cell and the accuracy column in Table 1 reads as both. On LongBench-v2 medium dense itself answers only 0.16–0.41 of items correctly (the benchmark is intentionally hard at 32K truncation), so raw accuracy is dominated by which questions the model can answer at all, not by which blocks the selector kept. Paired recall conditions on the dense-correct subset: when the sparse policy makes the same selections that let dense answer, paired is high; when it drops needed blocks, paired drops. The paired column in Table 2 therefore shows the same selector-quality ordering as accuracy but at higher contrast — e.g. on Qwen-14B accuracy is 0.19–0.21 across all sparse policies (within standard error of 0.027) while paired ranges 0.47–0.75. See Table 4 for the matched-budget decomposition of how much of the router lift is budget vs. selectivity.

VT. We do not include RULER VT (variable tracking, 3-hop) as a quality benchmark: dense itself fully solves $\leq 5/100$ items at 32K on every model we tested – Qwen-14B 5/100, Qwen-7B-1M 0/100, Nemo 0/100, and a smoke on the reasoning-tuned Qwen3.6-35B-A3B ($n = 30$) also gave 0/30 – which leaves the paired metric undefined or vanishingly thin on all but Qwen-14B. VT at hop = 3 with 3 distractor chains is harder than the current panel can handle at this parameter scale, including the MoE reasoning model; we defer VT to a future revision with a stronger reasoning base.

4.3 Speed–quality Pareto: 32K $\rightarrow$ 128K

Setup. We fix the model and vary the context length on the two panel models with training windows past 32K: Qwen2.5-7B-Instruct-1M (dense softmax) and Qwen3.6-35B-A3B (hybrid + MoE, QK-Norm). Configuration unchanged from §4.2 ($k_{\text{budget}} = 33$, $q = 0.40$, $\rho = 2$). Quality is RULER NIAH-multikey accuracy at $n = 100$; speed is per-prefill CUDA-event wall time at $n = 8$ (decode is the same dense SDPA path for every policy, so policy-dependent wall time comes purely from prefill).

Read. Two observations:

(i) *The winning backbone holds most of its accuracy as context grows; the losing one collapses.* Table 3 reports the winning backbone only – in the same data, the *losing* backbone on each model breaks: top- k

Table 3: Joint speed-quality Pareto on the *winning* backbone for each model (router-on-Quest for Qwen2.5-7B-1M, router-on-K-mean for Qwen3.6, from the QK-Norm reversal in §4.2). “router accuracy” is RULER NIAH-multikey at $n = 100$; “wall vs. dense” is the prefill-forward wall-time ratio at $n = 8$. Dense accuracy is 1.00 at every cell. Bold marks Pareto wins (router beats dense on speed at quality ≥ 0.80).

model	ctx	router accuracy	wall vs. dense	verdict
Qwen2.5-7B-1M	32K	1.00	$1.09\times$	ties quality, loses speed
	64K	0.92	0.87 $\times$	first Pareto win
	128K	0.81	0.62 $\times$	strong Pareto win
Qwen3.6-35B-A3B	32K	0.98	$1.12\times$	ties quality, loses speed
	64K	0.92	$1.00\times$	ties quality, ties speed
	128K	0.89	0.80 $\times$	first Pareto win

on K-mean falls $0.28 \rightarrow 0.09$ on Qwen2.5-7B-1M, and Quest on K-max falls $0.85 \rightarrow 0.50$ on Qwen3.6. The QK-Norm reversal from §4.2 is robust at every context, and the gap between winning and losing backbone widens with context on both architectures.

(ii) *Sparse runs slower than dense at short context and faster at long context; the crossover is structural (Amdahl)*. Attention’s share of dense prefill grows with context while the rest-of-model floor is invariant. On Qwen2.5-7B-1M attention shares are 31%/45%/63% at 32K/64K/128K, and the router crosses dense between 32K and 64K. On Qwen3.6 the attention share is much smaller (11%/19%/37%) because 30/40 layers are linear-attention rather than softmax, so the crossover shifts right, between 64K and 128K. The Pareto win materialises on both architectures; the context at which it activates is set by the architecture’s attention share.

4.4 Where the lift comes from: budget vs. selectivity

The router at $q = 0.40, \rho = 2$ spends on average 92.4 effective blocks per tile vs. 66 for Quest at $k_{\text{budget}} = 33 - \approx 1.4\times$ Quest’s budget. Some of the router’s lift over Quest could therefore come from spending the marginal budget *selectively* (the design intent), or simply from spending *more* budget on average. To separate the two we ran uniform Quest at a budget-matched control ($k_{\text{budget}} = 47$, eff. 94) and at intermediate budgets on the same RULER NIAH-multikey examples used in Table 1, on three models.

Read. The router’s lift over $\text{Quest}_{k_{\text{budget}}=33}$ splits into a *budget* term ($\text{Quest}_{k_{\text{budget}}=47}$ minus $\text{Quest}_{k_{\text{budget}}=33}$, what uniform expansion buys) and a *selectivity* term (router minus $\text{Quest}_{k_{\text{budget}}=47}$). The split is regime-dependent:

- *Qwen-14B (near-ceiling on its winning backbone)*: +5 pp total = +3 budget + +2 selectivity ($\approx 2/3$ budget, $1/3$ selectivity). Uniform Quest plateaus at 0.97 from $k_{\text{budget}} = 52$; the router lands one point above the asymptote – directionally consistent, inside noise at $n = 100$ (McNemar router vs. matched-Quest: 3 vs. 1 discordant, $p = 0.625$).
- *Nemo (off-ceiling)*: +15 pp = +6 budget + +9 selectivity ($\approx 2/5$ budget, $3/5$ selectivity). Uniform Quest does not plateau in the tested range, and the router at eff. 92.4 Pareto-dominates uniform $\text{Quest}_{k_{\text{budget}}=66}$ at eff. 132 by +6 pp – a separation no amount of uniform budget expansion within this range can close (one-sided sign test $p \leq 0.004$).
- *Qwen3.6 (near-ceiling on K-mean, the other backbone)*: on the K-max sweep the model looks like Qwen-14B (near-ceiling, budget-dominated), but K-max is the wrong backbone here – uniform Quest

Table 4: Budget-match ablation on RULER NIAH-multikey, ctx=32K, $n = 100$. Values are accuracy (= paired recall up to ≤ 0.01 on Nemo where dense = 0.96). Effective budget includes the sink and self-block forced into every kv_idx. Quest $_{k_{\text{budget}}=47}$ is the budget-matched control for the router at $q = 0.40$. Qwen-14B and Qwen3.6 are near the ceiling on their respective *winning* backbones; Nemo is off-ceiling.

policy	backbone	eff. budget	Qwen-14B	Nemo-12B	Qwen3.6
top- k	K-mean	66	0.51	0.29	0.94
Quest $k_{\text{budget}} = 33$	K-max	66	0.93	0.51	0.85
Quest $k_{\text{budget}} = 40$	K-max	80	0.95	0.53	0.88
Quest $k_{\text{budget}} = 47$ (matched)	K-max	94	0.96	0.57	0.93
Quest $k_{\text{budget}} = 52$	K-max	104	0.97	0.58	0.95
Quest $k_{\text{budget}} = 66$	K-max	132	0.97	0.60	0.98
router-on-Quest ($q = 0.40$)	K-max	92.4	0.98	0.66	0.97
router (K-mean) ($q = 0.40$)	K-mean	92.4	0.63	–	0.98

at $2\times$ eff. does not catch router-on-K-mean (0.98 at $1\times$ eff.). The Pareto domination on Qwen3.6 is across backbones, not across budgets.

Reading the three columns together: when uncertainty at the cutoff is rare (sharp scores), budget expansion dominates the lift; when it is common (diffuse scores), selective expansion dominates. The three columns bracket the regime in which the router operates.

When does the lift activate? LongBench-v1 as a negative control. The decomposition above answers *how* the router’s lift breaks down once it exists; it does not say *when* it exists at all. To pin that down we ran the LongBench-v1 panel (musique-2hop, musique-4hop, narrativeqa, qasper; $n = 30$, 32K context, $k_{\text{budget}} = 33$, $q = 0.40$). LB-v1 native prompt lengths are 5–30K, so truncation to 32K is essentially a no-op and the selector budget is generous relative to needle density. Result: dense, top- k , and router essentially tie on F1 across all four tasks; top- k already preserves 100% of dense-correct examples on musique-2hop, musique-4hop, and qasper – no headroom. One small lift on narrativeqa (router paired 1.00 vs. top- k 0.89, F1 +3.1 pp). **The router’s lift activates only when the per-query budget is small relative to where the answer-relevant evidence lives.** LB-v2 medium puts the budget under pressure (100K source $\rightarrow$ 32K window); LB-v1 does not.

4.5 Limitations

1. **Per-tile granularity.** Per-tile selection structurally loses needles that one row strongly wants but other rows in the tile ignore. The router recovers part of this gap but not all. A per-row scatter-into-bitmap kernel would close the rest of the gap without re-materialising the $[B, H, M, N_B]$ keep tensor; not yet built.
2. **Cross-architecture coverage.** The panel covers four models from three architecture classes (dense Qwen2.5, dense Mistral, hybrid+MoE Qwen3.6); broader coverage on additional families (Llama-3, MiniMax, Gemma) and additional standardised benchmarks (e.g. HELMET-RAG) would strengthen the cross-family claim.
3. **Engineering improvements.** Several speed optimisations remain on the table (a fused MoE kernel on hybrid+MoE; block-selection fused into the attention kernel for $\geq 256K$; per-row scatter for the per-tile granularity above). All are engineering work, not method changes; we leave them to future iterations.
4. **Hyperparameter sweeps incomplete.** The trigger fraction $q = 0.40$ and expansion factor $\rho = 2$

were selected on a RULER NIAH-multikey sweep at $n = 30$ and used unchanged everywhere; we did not re-sweep per task or per architecture. The diagnostic in Appendix D.5 opens a complementary path: *predicting* the Bayes-optimal q^* per task from the measurable noise scale rather than searching for it; this is not yet wired into the harness.

5. **What’s bounded, what’s owed.** Appendix D proves a non-asymptotic regret bound for the σ -quantile router vs. the Bayes-optimal expansion rule under Gaussian noise on block scores (Proposition 1, $C(\tau) \leq 2$). The bound is a *top- k identification* regret; converting it to an attention-output bound requires a softmax-sensitivity step that we sketch but do not prove (§D.6). Empirically the identification bound is $\sim 15\times$ looser than the observed router-vs-dense gap on RULER NIAH; the deferred composition is the largest source of slack.

Conclusion

We presented an uncertainty-gated value-of-information *router* for block-sparse attention selectors: a per-tile cutoff-margin signal σ triggers a $\rho\times$ expansion of the kept set on the bottom q -fraction of tiles per layer, independent of how block scores are computed and stackable with existing scoring backbones such as Quest.

The router delivers measurable accuracy gains across the panel. Headline: on LongBench-v2 medium ($n = 215$, $\text{ctx}=32\text{K}$) router-on-Quest paired recall reaches 0.75 vs. top- k 0.47 – +28 pp over the SSA-style baseline (McNemar $p < 0.01$). The lift reproduces on four models from three architecture classes. On Qwen2.5 (14B and 7B-1M) and Mistral-Nemo the K-max (Quest) backbone wins; the router lifts it by +15 pp on Nemo ($p < 0.001$), where Quest has paired headroom. On the QK-Norm hybrid + MoE Qwen3.6-35B-A3B the K-mean backbone wins instead: router-on-K-mean is the dominant policy, tracking accuracy $0.98 \rightarrow 0.92 \rightarrow 0.89$ across $32\text{K} \rightarrow 64\text{K} \rightarrow 128\text{K}$. The central composition claim – the router lifts whichever backbone wins on a given model – holds in every cell of the panel. At 128K the router preserves 0.81 and 0.89 of dense accuracy on Qwen2.5-7B-1M and Qwen3.6 respectively, while running at $0.62\times$ and $0.80\times$ dense wall time.

Three natural extensions: third-architecture validation (Llama-3 / MiniMax / Gemma) and HELMET-RAG; a fused MoE kernel on hybrid+MoE to push Qwen3.6’s 128K speed ratio toward $0.5\times$ dense; and composing the BAI top- k identification bound of Appendix D with an attention-output sensitivity step.

Code, results, and reproduction scripts:

<https://github.com/ThomasRossi/uncertainty-gated-block-sparse-attention>.

A Step-by-step derivation of σ and the trigger rule

This appendix walks through Section 3 at one equation per step, so a reader who has not followed the body can reproduce the implementation. Notation: B batch, H heads, M query rows, N keys, $N_B = N/\text{BLOCK}_N$ key blocks, $Q_t = M/\text{BLOCK}_M$ query tiles, $d_{\text{head}} = d_{\text{model}}/H$.

(A.1) Block-pooled keys.

$$\bar{k}_b^{(L,h)} = \frac{1}{\text{BLOCK}_N} \sum_{j \in \text{block } b} k_j^{(L,h)}, \quad b \in \{0, \dots, N_B - 1\}. \quad (8)$$

(A.2) Per-row block scores.

$$s^{(L)}[i, h, b] = \frac{q_i^{(L,h)} \cdot \bar{k}_b^{(L,h)}}{\sqrt{d_{\text{head}}}}, \quad i \in \{0, \dots, M - 1\}. \quad (9)$$

(A.3) Per-tile reduction.

For Q-tile t covering rows $\{t\text{BLOCK}_M, \dots, (t+1)\text{BLOCK}_M - 1\}$,

$$\text{tile_score}[t, h, b] = \max_{r \in \text{tile } t} s[r, h, b]. \quad (10)$$

We use max-pool because dropping a block the most aggressive row wants is worse than dropping one only weak rows want; mean-pool was tested and underperforms.

(A.4) Forcing the sink and self-block. Let $b_{\text{sink}} = 0$ and $b_{\text{self}}(t) = \lfloor t\text{BLOCK}_M/\text{BLOCK}_N \rfloor$ (the key block containing the tile’s own rows). We add $+\infty$ to the corresponding entries of tile_score before the top- k :

$$\widetilde{\text{tile_score}}[t, h, b] = \text{tile_score}[t, h, b] + \infty \cdot \mathbb{I}[b \in \{b_{\text{sink}}, b_{\text{self}}(t)\}]. \quad (11)$$

This guarantees both blocks land in the kept set without a separate concat-and-dedup.

(A.5) Top- $(k+1)$ and the sorted prefix.

We compute

$$\text{idx}_{\text{sorted}}[t, h, \cdot], \quad \widetilde{s}_{\text{sorted}}[t, h, \cdot] = \text{top-}(k+1)(\widetilde{\text{tile_score}}[t, h, \cdot]), \quad (12)$$

in descending order. The first k indices are $\text{kv_idx}[t, h]$; the $(k+1)$ -th score is what we need for σ .

(A.6) Normalised cutoff margin.

$$\sigma[t, h] = \frac{\widetilde{s}_{\text{sorted}}[t, h, k-1] - \widetilde{s}_{\text{sorted}}[t, h, k]}{\widetilde{s}_{\text{sorted}}[t, h, 0] - \widetilde{s}_{\text{sorted}}[t, h, k]}. \quad (13)$$

Edge cases: if the denominator is zero (all top scores tied) we set $\sigma = 0$ (treat as maximally uncertain); $+\infty$ entries from (A.4) participate in the sorted prefix on top, so $\widetilde{s}_{\text{sorted}}[t, h, 0]$ is always $+\infty$ – in practice we therefore exclude the forced indices from the σ computation and apply Eq. (A.6) to the remaining tile scores.

(A.7) Head aggregation.

$$\bar{\sigma}[t] = \frac{1}{H} \sum_{h=1}^H \sigma[t, h]. \quad (14)$$

(A.8) Per-layer empirical quantile.

$$\tau^{(L)} = \text{quantile}_q(\bar{\sigma}[\cdot]), \quad \text{trigger}[t] = \mathbb{1}[\bar{\sigma}[t] \leq \tau^{(L)}]. \quad (15)$$

(A.9) Padded expansion. Working budget k_{budget} , expansion factor ρ , sentinel $b_{\perp} = N_B$. We allocate $\text{kv_idx}_{\text{final}} \in \mathbb{Z}^{B \times H \times Q_t \times \rho k_{\text{budget}}}$ and fill

$$\text{kv_idx}_{\text{final}}[t, h] = \begin{cases} [\text{idx}_{\text{sorted}}[t, h, 0], \dots, \text{idx}_{\text{sorted}}[t, h, \rho k - 1]] & \text{trigger}[t] = 1, \\ [\text{idx}_{\text{sorted}}[t, h, 0], \dots, \text{idx}_{\text{sorted}}[t, h, k - 1], b_{\perp}, \dots, b_{\perp}] & \text{otherwise.} \end{cases} \quad (16)$$

The triggered case requires a top- ρk rather than a top- $(k+1)$, which we batch with the top- $(k+1)$ used for σ as a single top-max($\rho k, k+1$) = ρk when $\rho \geq 2$.

(A.10) Kernel iteration. The Triton kernel’s inner loop iterates $\text{kv_idx}_{\text{final}}[t, h]$ and skips any entry equal to $b_{\perp}$ on a single compare. The online softmax is unchanged from FlashAttention.

Average attended budget. A row in tile t attends to k blocks if $\text{trigger}[t] = 0$ and ρk if $\text{trigger}[t] = 1$. The expected per-row attended budget is therefore

$$\mathbb{E}[\text{attended}] = k_{\text{budget}} \cdot (1 + q(\rho - 1)), \quad (17)$$

which at $q = 0.4, \rho = 2$ is $1.4 k_{\text{budget}}$. This is the headline cost of the router.

B Pointer-Chase Haystack: construction and diagnostic results

The Pointer-Chase Haystack (PCH) is a custom diagnostic benchmark we built during method development to isolate selector quality from model capability. We retain it here for transparency on the development process; the headline empirical results in Section 4 use the standardised benchmarks RULER NIAH and LongBench-v2 medium.

Design goals. We needed a task that (i) has a dense-attention ceiling of $\approx 100\%$, so failures are unambiguously selector failures, and (ii) is *query-latent*, meaning the relevant tokens cannot be identified by surface matching against the query alone. RULER NIAH multi-key satisfies (i) but not (ii); RULER VT satisfies (ii) but not (i) (Qwen 7B fails it).

Construction. A pointer chase of depth h is a sequence of indexed entries

Entry ID₀: continue at entry ID₁.
Entry ID₁: continue at entry ID₂.
 $\vdots$
Entry ID_h: the recorded value is V .

with all ID_k drawn from a private namespace. A PCH instance contains one gold chain plus D distractor chains, all entries shuffled into a haystack of target token length N . The query gives ID_0 and asks for V . Hop 0 is query-identifiable; hops $1, \dots, h$ are query-latent. Compounding is by construction: missing entry k permanently blocks every entry $k+1, \dots, h$.

Grounding. The pointer chase is the canonical communication-complexity problem for unavoidable k -round sequential dependency: it provably cannot be collapsed into fewer rounds. Every per-hop operation is trivial (follow a link, read a value), keeping a capable dense model at the $\sim 100\%$ ceiling.

Metric. We report *hit rate*: the fraction of gold-chain blocks the selector keeps in `kv_idx`. Hit rate is well-defined only when the gold blocks are known by construction, which is the case here.

Result. Table 5 reports hit rate at $h = 3$, fixed $k_{\text{budget}} = 40$ blocks/row (32K, 64K) and 29 (8K), on Qwen2.5-14B-Instruct.

Table 5: PCH hop=3 hit rate. Hit rate is the fraction of gold-chain blocks the selector keeps. Steady-state wall time is reported for orientation; the main efficiency analysis is in Section 4.3.

ctx	policy	dt (steady)	vs dense	hit rate
8K	dense	1.4s	1.00×	1.00
8K	top- k (per-tile)	1.6s	1.14× slower	0.79
8K	router q=0.10	1.6s	1.14× slower	1.00
32K	dense	6.9s	1.00×	1.00
32K	top- k	6.05s	0.88× (12% faster)	0.63
32K	router q=0.10	6.6s	0.96× (4% faster)	0.78
64K	dense	17.5s	1.00×	1.00
64K	top- k	14.1s	0.81× (1.24× faster)	0.42
64K	router q=0.10	14.8s	0.85× (1.18× faster)	0.54

Router lifts hit rate over plain top- k by +12–+15 pp across scales (and recovers dense’s 1.00 at 8K). The gain is monotonic with context length, consistent with the structural cost of per-tile selection increasing with N that motivates the router in the first place.

C Per-row vs. per-cell aggregation: why heads are pooled before thresholding

We tested two natural alternatives to Eq. (6) and they both fail. We report them here so the design choice in Section 3 can be understood as a tested decision rather than an arbitrary one.

Per-cell quantile. Trigger on $\mathbb{K}[\sigma[t, h] \leq \text{quantile}_q(\sigma[\cdot, \cdot])]$, i.e. flag the bottom- q *cells*, not tiles. Result: the rescue set is dominated by per-head idiosyncratic noise; the gain over plain top- k vanishes for every q we tested. Mechanism: heads in the same tile are highly correlated (PCH cross-head agreement matches peakedness byte-for-byte across candidates), so per-head σ values carry mostly the same signal plus head-private noise; quantile-by-cell amplifies the noise.

Absolute threshold τ on σ . Trigger on $\mathbb{K}[\sigma[t, h] \leq \tau]$ for a fixed τ . Result: works at one layer’s σ distribution but not across the stack; we observe systematic distribution shift early-vs-late layers. A per-layer quantile is the simplest layer-conditional fix and dominated the absolute- τ sweep.

Row-grain q sweep. On PCH at 32K, row-grain $q = 0.10$ reaches near-top- k effective budget at hit rate 0.78, vs. plain top- k at 0.63. On RULER NIAH-multikey at $n = 30$, ctx=32K (fixed $k_{\text{budget}} = 33$, kernel-v2, row-grain quantile router), we swept $q \in \{0.10, 0.20, 0.30, 0.40\}$ and observed paired recall 0.38, 0.35, 0.47, 0.53 respectively, vs. top- k baseline 0.35. The trend is monotonic in q over the tested range and plateaus near $q = 0.40$. We use $q = 0.40$ for the headline results on RULER and LB-v2; the regime where q should differ across tasks (because the budget-vs-evidence Pareto shifts) is consistent with the smaller value chosen for PCH, but a per-task sweep on LB-v2 medium remains open.

D σ as a best-arm-identification exploration index

The Method section introduces σ as a value-of-information proxy and validates it empirically. This appendix gives the formal anchor: under a Gaussian noise model on the per-block relevance signal, σ is a dimensionless exploration index in the sense of best-arm identification (BAI), and the σ -quantile router is $C(\tau)$ -times optimal for the per-tile expansion decision relative to the Bayes-optimal rule, with explicit non-asymptotic constants and a measurable leading factor.

D.1 The BAI intuition that motivates σ

The classical fixed-confidence top- k identification problem of Kaufmann et al. (2016); Garivier and Kaufmann (2016) asks for $\hat{S}$ such that $\mathbb{P}(\hat{S} \neq S^*(\mu)) \leq \delta$ using as few arm pulls as possible. The asymptotically optimal Track-and-Stop algorithm pulls $b^* = \arg \max_b [w_b^*(\hat{\mu}) - T_b/T]$ where the optimal allocation w^* solves a max-min programme whose minimiser concentrates on the cutoff pair $(k-1, k)$: any alternative world in which the top- k differs from $\hat{\mu}$'s is achieved most cheaply by perturbing those two arms. Under sub-Gaussian noise, the per-step exploration value at the cutoff scales as

$$V_{\text{TaS}}(\hat{\mu}) \propto (\hat{\mu}_{(k-1)} - \hat{\mu}_{(k)})^{-2}, \quad (18)$$

since KL divergence between Gaussians of equal variance scales as the squared mean gap. **Small empirical gap at the cutoff $\iff$ high exploration value $\iff$ allocate budget here.**

Our setting differs from classical BAI in three ways: (a) we observe one sample per arm per tile, not a sequence; (b) the budget decision is binary per tile (expand to ρk or keep at k), not a continuous allocation; (c) we do not know τ . The dimensionless normalisation $\sigma_t = g_t/r_t$ – with $g_t := s_{t,(k-1)} - s_{t,(k)}$ and $r_t := s_{t,(0)} - s_{t,(k)}$ – replaces the missing τ with the per-tile spread, on the heuristic that the spread concentrates around its layer-conditional mean and absorbs τ . An absolute-threshold rule (Appendix C) fails because the marginal distribution of s shifts across layers; σ is the simplest layer-conditional and tile-conditional normaliser.

The rest of this appendix turns this heuristic into a regret bound.

D.2 Setting and policies

For each Q-tile $t \in [T]$ (we suppress the head index throughout) and block $b \in [N_B]$,

$$s_{t,b} = \mu_{t,b} + \eta_{t,b}, \quad \eta_{t,b} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \tau^2), \quad (19)$$

with $\mu_t \in \mathbb{R}^{N_B}$ deterministic conditional on the tile. We assume a uniform top- k gap $\mu_{t,(k-1)} - \mu_{t,(k)} \geq \Delta_{\min} > 0$, where $\mu_{t,(j)}$ denotes the j th order statistic. Let $\hat{S}_m(s_t) = \{b : s_{t,b} \geq s_{t,(m)}\}$ be the empirical top- m set under s_t .

A *policy* Π_q chooses a subset $E \subseteq [T]$ with $|E|/T \rightarrow q$ to expand, outputs $\widehat{S}_{\rho k}(s_t)$ for $t \in E$ and $\widehat{S}_k(s_t)$ otherwise. The per-tile identification error is $\mathcal{E}_t(\widehat{S}) := \mathbb{P}(\widehat{S} \not\supseteq S^*(\mu_t) \mid s_t)$, where $S^*(\mu_t)$ is the true top- k . The per-tile *expansion value* is

$$V_t^*(s_t) := \mathcal{E}_t(\widehat{S}_k(s_t)) - \mathcal{E}_t(\widehat{S}_{\rho k}(s_t)). \quad (20)$$

The Bayes-optimal expansion policy under the budget constraint is $\Pi_q^* := \{t : V_t^* \geq V_{(q)}^*\}$ (Neyman–Pearson on V^*). The router we use is the σ -quantile policy $\Pi_q^\sigma := \{t : \sigma_t \leq \sigma_{(q)}\}$.

D.3 Theorem and proof

Proposition 1 (σ -quantile regret bound). *Under model (19) with $\Delta_{\min} > 0$ and a bounded density of g_t in a neighbourhood of its q -quantile $g_{(q)}$,*

$$\mathcal{E}(\Pi_q^\sigma) - \mathcal{E}(\Pi_q^*) \leq 2 \text{CV}(r) \cdot q(1-q) \cdot \Phi\left(-\frac{g_{(q)}}{\sqrt{2}\tau}\right) \cdot \frac{r_q}{\tau} + O(T^{-1/2}), \quad (21)$$

where r_q is the q -quantile of r_t across tiles and $\text{CV}(r) := \sqrt{\text{Var}_t r_t} / \mathbb{E}_t r_t$. For sub-Gaussian noise with variance proxy τ and true Gaussian variance $\sigma^2 \leq \tau^2$, multiply the leading constant by $(1 + \kappa)$, $\kappa := (\tau/\sigma)^2 - 1$.

The proof composes four steps. Each step is the closure of a hand-wave in the earlier sketch of this appendix.

Step 1 – cutoff-pair localisation. Under flat prior, $\mu_t \mid s_t \sim \mathcal{N}(s_t, \tau^2 I)$. The event $\widehat{S}_k(s_t) \neq S^*(\mu_t)$ is a union of swap events between arms in and out of $\widehat{S}_k$; Bonferroni with Gaussian tail suppression gives

$$\mathcal{E}_t(\widehat{S}_k) = \Phi\left(-\frac{g_t}{\sqrt{2}\tau}\right) (1 + \delta_1), \quad |\delta_1| \leq (N_B - 2) \exp\left(-\frac{(s_{(k)} - s_{(k+1)}) g_t}{2\tau^2}\right), \quad (22)$$

since any pair (b, b') other than the cutoff $(k-1, k)$ has $s_b - s_{b'} \geq s_{(k-1)} - s_{(k+1)} = g_t + (s_{(k)} - s_{(k+1)})$, and the Gaussian-tail ratio $\Phi(-x - a)/\Phi(-x) \leq e^{-ax - a^2/2}$ suppresses it exponentially whenever the secondary gap is $\Omega(\tau)$. The top- k gap assumption $\Delta_{\min} > 0$ ensures this in expectation. Applying the same at rank ρk ,

$$V_t^*(s_t) = \Phi\left(-\frac{g_t}{\sqrt{2}\tau}\right) - \Phi\left(-\frac{g_t^{(\rho)}}{\sqrt{2}\tau}\right) + O(\delta), \quad g_t^{(\rho)} := s_{(\rho k-1)} - s_{(\rho k)}. \quad (23)$$

The single scalar g_t is therefore a near-sufficient statistic for V_t^* (modulo $g_t^{(\rho)}$, controlled next).

Step 2 – monotone spacings (expansion never harms in expectation). The spacing density of the j th gap among N_B iid Gaussians is monotone-increasing in $\min(j, N_B - j)$ (David and Nagaraja, 2003, Ch. 3). Hence for $\rho k \leq N_B/2$, $\mathbb{E}[g_t^{(\rho)} \mid g_t] \geq g_t$ by stochastic ordering, so $V_t^* > 0$ in expectation and is monotone-decreasing in g_t . (The condition $\rho k \leq N_B/2$ holds in all our experiments: with $\rho = 2$, $k = 33$, $N_B \in \{128, 256, 512\}$.)

Step 3 – σ tracks g up to spread variation. Write $r_t = \bar{r}(1 + \xi_t)$ with $\bar{r} := \mathbb{E}_t r_t$, $\mathbb{E} \xi_t = 0$, $\text{Var} \xi_t = \text{CV}(r)^2$. Then

$$\sigma_t = \frac{g_t}{\bar{r}(1 + \xi_t)} = \frac{g_t}{\bar{r}} (1 - \xi_t + O(\xi_t^2)). \quad (24)$$

Lemma 1 (Rank perturbation). *The fraction of pairs (t, t') ranked inconsistently by g_t and σ_t is at most $2\text{CV}(r)$.*

Proof. A pair is swapped iff $g_t < g_{t'}$ but $g_t(1 - \xi_t) > g_{t'}(1 - \xi_{t'})$. Rearranging, $g_{t'} - g_t < g_{t'}\xi_{t'} - g_t\xi_t \leq |\xi_{t'}|g_{t'} + |\xi_t|g_t$. Taking probabilities and applying Markov gives $\mathbb{P}(\text{swap}) \leq 2\mathbb{E}|\xi_t| \leq 2\sqrt{\mathbb{E}\xi_t^2} = 2\text{CV}(r)$. $\square$

Step 4 – regret from the boundary band. Define the boundary band $\mathcal{B}_q := \{t : g_t \in [g_{(q)} \pm \delta]\}$. Outside $\mathcal{B}_q$ the σ -perturbation in (24) is too small to cross the quantile boundary, so both rules agree. Choose $\delta = \text{CV}(r) \cdot g_{(q)}$: under the assumed bounded density of g_t near $g_{(q)}$, $|\mathcal{B}_q|/T = O(\text{CV}(r))$. Each misclassification in $\mathcal{B}_q$ costs at most $|V_t^* - V_{(q)}^*|$, bounded by the mean-value theorem on (23):

$$|V_t^* - V_{(q)}^*| \leq \phi\left(-\frac{g_{(q)}}{\sqrt{2}\tau}\right) \cdot \frac{\delta}{\sqrt{2}\tau}.$$

Combining gives the leading term of (21), with $g_{(q)} \leq r_q$ absorbed into the r_q/τ factor and a numerical constant of $1/\sqrt{\pi}$ folded into the leading 2. The $O(T^{-1/2})$ term comes from replacing the population q -quantile of V^* by its empirical quantile across T tiles, controlled by the Dvoretzky–Kiefer–Wolfowitz inequality (Dvoretzky et al., 1956; Massart, 1990). $\square$

D.4 The constant $C(\tau) \leq 2$

The Gaussian-noise bound $C(\tau) \leq 2$ tracks the worst-case ratio of r_t across tiles. Under (19), standard Gaussian extreme-value concentration gives $r_t \in [(\mu_{(0)} - \mu_{(k)}) \pm \tau\sqrt{2\log N_B}]$ with probability $1 - O(N_B^{-1})$. Hence $\sup_t r_t / \inf_t r_t \rightarrow 1$ as $\Delta_{\min}/\tau \rightarrow \infty$, and is bounded by 3 in the worst-case signal-noise-comparable regime $\mu_{(0)} - \mu_{(k)} \asymp \tau\sqrt{2\log N_B}$. Anchoring the σ -rule’s quantile boundary on the layer-conditional median of r (rather than its extreme) yields the symmetric bound $C(\tau) \leq 2$.

For sub-Gaussian noise with proxy τ and true Gaussian variance σ^2 , the extreme-value scale inflates to $\tau\sqrt{2\log N_B}(1 + \kappa)$ where $\kappa = (\tau/\sigma)^2 - 1$, hence $C(\tau) \leq 2(1 + \kappa)$. This is the precise content of the earlier “linear in the sub-Gaussian variance proxy” claim.

D.5 Empirical estimation of the bound

The bound (21) is non-asymptotic and every quantity on the right-hand side is measurable from a small diagnostic run. The recipe is:

1. For each layer ℓ and head h , dump the per-tile block-score vector $(s_{t,b})_{b \in [N_B]}$ during prefill on a small example panel ($n = 10$ is sufficient: a 32K context with $\text{BLOCK}_M = 64$ yields ~ 512 Q-tiles per (layer, head, example), so ~ 5000 tiles per (layer, head) overall – ample for sample mean and variance of r).
2. Compute $r_t, g_t, \sigma_t, g_t^{(\rho)}$ from the sorted scores and aggregate $\text{CV}(r), r_q, g_{(q)}$ per layer.
3. Estimate τ as the residual standard deviation of $s_{t,b}$ against a smoothed reference – the simplest being the across-tile median of $s_{\cdot,b}$ for each block b , on the heuristic that the true $\mu_{t,b}$ has slow tile-to-tile variation while $\eta_{t,b}$ is iid.
4. Plug into (21). The bound becomes a per-layer numerical regret; aggregating by averaging or worst-casing across layers gives a single end-to-end number to compare against the empirical paired-recall gap (e.g. the 2 pp router-vs-dense gap on RULER NIAH n=100).

5. Cross-check Lemma 1 empirically: scatter $g_t^{(\rho)}$ against g_t ; the conditional-mean line should lie above the diagonal.

Measured bound on Qwen-14B, RULER NIAH n=10, 32K. The diagnostic collects $\sim 5.7\text{M}$ (B, H, Q_t) tile cells per scoring backbone (10 examples $\times$ 48 layers $\times$ 40 heads $\times$ ~ 512 tiles, masked to causally-valid tiles with $\geq \rho k + 1$ visible blocks) plus, per call, a $[B, H]$ noise-scale estimate $\hat{\tau}$ from the first-difference dispersion of per-row block scores along the query dimension (smoothness-residual estimator: under $\mu_{t+1,b} \approx \mu_{t,b}$ for adjacent rows of the prompt, $\mathbb{E}[(s_{t+1,b} - s_{t,b})^2] = 2\tau^2$). All quantities aggregated over the 48 Qwen-14B layers and 40 heads:

backbone	$\overline{\text{CV}(r)}$	$\overline{r_q}$	$\langle g_t^{(\rho)} - g_t \rangle$	$\hat{\tau}$ (median)	lead	bound($\hat{\tau}$)
mean (topk)	0.38	4.06	$+4.8 \times 10^{-4}$	0.94	0.74	0.39
Quest	0.32	8.50	$+3.0 \times 10^{-3}$	2.25	1.30	0.28
router-on-Quest	0.32	8.63	$+2.8 \times 10^{-3}$	2.24	1.31	0.28

where $\text{lead} := 2 \text{CV}(r) q(1-q) r_q$ is the τ -free leading factor at $q = 0.40$, and $\text{bound}(\hat{\tau}) := \text{lead} \cdot \Phi(-g_q^+/\sqrt{2\hat{\tau}})/\hat{\tau}$ with g_q^+ the 40th-percentile of g_t on the informative subset $\{t : g_t > 0\}$ (the **bf16** floor for Quest scores yields exact zeros on $\sim 83\%$ of tiles where the cutoff lands in the flat tail; conditioning on $g_t > 0$ is the honest active-set restriction). Three observations.

(i) *Lemma 1 holds empirically on every layer.* $\mathbb{E}[g_t^{(\rho)} - g_t] > 0$ for every (layer, backbone) cell ($48 \times 3 = 144$ cells, no exceptions) – the expansion-helps-in-expectation guarantee survives the actual block-score distribution on a real transformer, not just iid Gaussian order statistics.

(ii) *Noise-dominated regime.* $g_q^+/\hat{\tau} \approx 0.03$ for both backbones, so $\Phi(-g_q^+/\sqrt{2\hat{\tau}}) \approx 0.49$ – the cutoff is buried in the noise. Sensitivity at $\hat{\tau}/2$ and $2\hat{\tau}$ is flat: the bound varies within a factor of ~ 2.7 (Quest: 0.55, 0.28, 0.14 at $\hat{\tau}/2, \hat{\tau}, 2\hat{\tau}$ respectively), because Φ saturates at 0.5 and the $1/\hat{\tau}$ scaling dominates. This is the regime where the BAI bound has the least Bonferroni slack but also the smallest informational gap between the σ -rule and any other rule – both are near-coin-flip on the contested boundary.

(iii) *The bound is $\sim 15\times$ looser than the empirical gap, and we can name the looseness.* Empirical router-vs-dense identification gap on RULER NIAH $n = 100$ is ~ 2 pp (router-on-Quest 0.98 vs. dense 1.00); the bound predicts ≤ 28 pp on the same backbone. The $\sim 15\times$ gap decomposes as: (a) Lemma 2’s $2 \text{CV}(r)$ rank-swap fraction is a Markov-inequality upper bound, typically loose by $2\text{--}3\times$ for Gaussian-tail order statistics; (b) the leading factor sums regret over the full boundary band $|\mathcal{B}_q|/T = O(\text{CV}(r))$, but actual swaps within $\mathcal{B}_q$ are sparser than the worst case; (c) identification error overstates attention-output error because most mis-identified blocks at the cutoff carry negligible softmax mass (the still-deferred Step in §D.6). The first two are tightenable inside this proof; the third is the natural follow-up theorem.

Diagnostic produced by `dump_block_scores.py` (cache-safe runtime monkey-patch; lives outside `code_version`) and `analyze_block_scores.py`. Raw pickle and per-layer table available with the release. Total diagnostic cost: $\sim \$2$ across the two $n=10$ runs.

D.6 What (21) does *not* bound

The proposition controls top- k *identification* error – the probability that the kept set $\hat{S}$ misses an element of $S^*(\mu)$. The downstream quantity is the *attention-output* error,

$$\|o^{\hat{S}} - o^{\text{full}}\|_2 \leq \left(\sum_{b \notin \hat{S}} \alpha_b^*\right) \cdot \max_j \|v_j\|,$$

where α^* is the dense softmax. The dropped softmax mass is itself Φ -tailed in the cutoff gap – the Step-1 argument applied at the softmax temperature $1/\sqrt{d}$ instead of τ – so composing gives a bound proportional to $\Phi(-g_t/\tau_{\text{softmax}}) \cdot \max_j \|v_j\|$. The two temperatures are different constants and the composition is one paper of additional work; we record it as the natural follow-up theorem.

Bought. A non-asymptotic regret bound for the σ -quantile router, with explicit constants, a measurable leading factor $\text{CV}(r) \cdot r_q/\tau$, and $C(\tau) \leq 2$ under Gaussian noise.

Owed. The composite attention-output bound (one further composition with a softmax-sensitivity inequality); empirical $\text{CV}(r)$ and τ per layer (a small diagnostic run, plan in §D.5); and a treatment of LB-v2 medium that distinguishes “the bound is loose” from “the budget is binding” (budget sweep, Section 4.5).

References

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. <https://arxiv.org/abs/1706.03762>.
- J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *MLSys*, 2024. <https://arxiv.org/abs/2406.10774>.
- Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. <https://arxiv.org/abs/2306.14048>.
- Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024. <https://arxiv.org/abs/2404.14469>.
- H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *NeurIPS*, 2024. <https://arxiv.org/abs/2407.02490>.
- J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv preprint, 2025. <https://arxiv.org/abs/2502.11089>.
- E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, et al. MoBA: Mixture of block attention for long-context LLMs. arXiv preprint, 2025. <https://arxiv.org/abs/2502.13189>.
- Subquadratic. How SSA makes long-context practical. 2025. <https://subq.ai/how-ssa-makes-long-context-practical>.

- T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022. <https://arxiv.org/abs/2205.14135>.
- C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What’s the real context size of your long-context language models? In *COLM*, 2024. <https://arxiv.org/abs/2404.06654>.
- Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *ACL*, 2024. <https://arxiv.org/abs/2308.14508>.
- Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint, 2024. <https://arxiv.org/abs/2412.15204>.
- H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. arXiv preprint, 2024. <https://arxiv.org/abs/2410.02694>.
- A. Garivier and E. Kaufmann. Optimal best arm identification with fixed confidence. In *COLT*, 2016.
- E. Kaufmann, O. Cappé, and A. Garivier. On the complexity of best-arm identification in multi-armed bandit models. *JMLR*, 17(1):1–42, 2016.
- H. A. David and H. N. Nagaraja. *Order Statistics*. Wiley, 3rd edition, 2003.
- A. Dvoretzky, J. Kiefer, and J. Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *Annals of Mathematical Statistics*, 27(3):642–669, 1956.
- P. Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *Annals of Probability*, 18(3):1269–1283, 1990.